

Module 6 — Claude Agent SDK

Lesson 6.3

Tool use patterns

Built-in tools cover most file/shell work. Define custom tools when you need to wire in your own systems.

Why it matters

The SDK ships with the same tools Claude Code uses — Read, Write, Edit, Glob, Grep, Bash, Agent — and they're enough for most coding-style tasks. But the moment your agent needs to call your API, query your DB, or push to your queue, you'll want a custom tool.

There are two ways to add custom tools: via an MCP server, or directly registered with the SDK. Pick by lifecycle.

Path 1 — MCP server tools

If the tool is reusable across clients (Claude Code + your agents + your IDE), put it in an MCP server. The SDK can attach to MCP servers the same way Claude Code does:

```
options = ClaudeAgentOptions(
    mcp_servers={
        "linear": {
            "command": "npx",
            "args": ["-y", "@modelcontextprotocol/server-linear"],
            "env": {"LINEAR_API_KEY": os.environ["LINEAR_API_KEY"]},
        },
    },
)
```

Now the agent can call `mcp__linear__create_ticket` etc. — same names, same behavior as in Claude Code.

When to use: the tool is general-purpose and you want one implementation. Covered in Module 5.

Path 2 — Inline custom tools (SDK-only)

If the tool is one-off for this agent and doesn't need to exist outside it, define it inline. The SDK supports tool definitions similar to the raw Anthropic API:

```
from claude_agent_sdk import tool

@tool(
    name="query_orders",
    description="Run a parameterized read-only query against the orders DB.",
)
def query_orders(sql: str, params: list | None = None) -> dict:
    rows = db.execute(sql, params or []).fetchall()
    return {"rows": rows, "count": len(rows)}

options = ClaudeAgentOptions(
    tools=[query_orders],
    ...
)
```

(API may vary slightly — confirm against current SDK docs.)

When to use: the tool is internal-only to this script, or you're prototyping and don't want the overhead of an MCP server yet.

Tool description = tool quality

For both paths, the description Claude sees is what determines whether and how it uses the tool.

Bad descriptions cause:

- Claude not calling the tool when it should
- Claude calling the tool with wrong arguments
- Claude calling the wrong tool when two are similar

The contract: be precise about what the tool does, when to use it, what it returns, what side effects it has.

```
@tool(
    name="post_to_slack",
    description="""
    Post a message to a Slack channel.

    Use when the user explicitly asks to notify Slack, or when an agent run
    completes and the configured destination is Slack.

    Args:
        channel: Channel ID (NOT name). Use the `list_slack_channels` tool to discover.
        text: Markdown-formatted body.

    Returns:
        {"ts": str, "channel": str} - Slack's message timestamp and channel ID

    Side effects: posts a real message. There is no draft mode.
    """,
)
def post_to_slack(channel: str, text: str) -> dict: ...
```

Notice it tells the model:

- What the tool does
- When to call it (and importantly, when NOT to)
- Argument shape (with a hint that names aren't valid)
- What it returns
- That it has side effects

The tool-use loop

The SDK manages the loop. In pseudo-code:

```
while not done:
    response = model(messages, tools)
    if response wants to call a tool:
        result = run_tool(response.tool_call)
        messages += [response, tool_result(result)]
    else:
        messages += [response]
        done = True
```

You don't write this loop; the SDK does. What you can do is intercept:

- `can_use_tool` callback: approve, deny, or modify each tool call before execution
- Event stream: observe tool calls and results in real time
- `permission_mode`: blanket policy (default / `acceptEdits` / `bypassPermissions` / `plan`)

Error handling

Tools can fail. The SDK reports failures back to the model as part of the conversation, and the model decides what to do — often retry, sometimes give up. Two practices help:

1. Raise informative errors

```
@tool(...)
def query_orders(sql: str) -> dict:
    if "DELETE" in sql.upper() or "UPDATE" in sql.upper():
        raise ValueError("This tool is read-only. Use the appropriate write tool.")
    ...
```

The error message reaches the model. Make it actionable.

2. Bound retries

If the model gets stuck retrying the same broken tool call, the loop wastes turns and money. Use `max_turns` to cap and inspect the result:

```
options = ClaudeAgentOptions(max_turns=15, ...)

# Inspect the final result
async for event in query(prompt=task, options=options):
    if isinstance(event, ResultMessage):
        if event.stop_reason == "max_turns":
            # Task didn't complete — log, alert, fail loudly
            raise TaskIncompleteError(event)
```

When to use built-in vs. custom tools

Task	Tool
Read a file	Built-in <code>Read</code>
Edit a file	Built-in <code>Edit</code>
Run a shell command	Built-in <code>Bash</code> (scope with <code>allowed_tools</code> patterns)
Search code	Built-in <code>Grep</code> / <code>Glob</code>
Query your DB	Custom (MCP or inline)
Call your API	Custom
Post to Slack	Custom

If you find yourself wrapping a shell command in a custom tool because Bash is "too dangerous," that's a sign your permissions need tightening, not that you need a new tool. Use

```
allowed_tools=["Bash(curl:*)", "Bash(jq:*)"] style scoping.
```

Try it

- 1 Add an inline custom tool to the example agent: `get_current_time()` returns ISO timestamp.
- 2 Run a task that should use it: `"what time is it, and write that to a file in this dir"`. Notice the model calling it.
- 3 Add a deliberately broken tool that raises. Watch the model react.
- 4 Add an MCP server (the notes server from example 04) to the agent. Use a task that exercises it.

Gotchas

- **Tool names and shapes are stable contracts.** Renaming a tool the agent is mid-task on breaks recovery. Version your tool names.
- **Don't accept raw SQL/shell from tool args without validation.** The model is the caller; prompt-injected content can flow through.
- **Tool results are tokens.** A tool returning a 1 MB JSON blob blows up context. Truncate, summarize, or paginate at the tool boundary.
- **MCP servers add latency.** A custom inline tool calling the same backend is faster (no subprocess + JSON-RPC round trip).

Further reading

- 6.4 Prompt caching — keep tool definitions cached
- 5.3 Build a Python MCP server
- SDK docs: <https://docs.claude.com/en/docs/agents/agent-sdk>

Next

→ 6.4 Prompt caching for cost and latency